

How the “Text Report” Field is Generated

A walkthrough of `btreport/vasari_features/vasari_auto_v2.py` with a real BraTS2021_01505 example

1. Overview

The **Text Report** field is a one-sentence, plain-English summary of the most salient VASARI features for a subject. It is the only narrative-style field in the metadata JSON and is intended both as a quick human-readable snapshot and as a compact input that can be fed to downstream LLM prompts.

Although `generate_report.py` is the orchestrator, the string itself is constructed inside the **ExtractVASARI** class in `btreport/vasari_features/vasari_auto_v2.py` (specifically the `__call__` method, line 735). The orchestrator only merges the returned dictionary into metadata via `metadata.update(vasari_summary)`.

2. End-to-end Pipeline

The Text Report is produced by seven micro-steps inside a single call to the extractor. The numbered diagram below mirrors the source ordering:

#	Step	Where in code
1	Instantiate ExtractVASARI and call it from step [3/5]	<code>generate_report.py</code> L145-146
2	Compute raw VASARI features (numeric codes + raw strings)	<code>extract_vasari_features()</code> L543-714
3	Translate codes → readable phrases with <code>vasari_concise</code> map	<code>__call__</code> L718
4	Filter to 8 whitelisted feature keys	<code>__call__</code> L719-733
5	Re-translate full report with <code>vasari</code> map (for output dict)	<code>__call__</code> L734
6	Build sentence: join + capitalize + period	<code>__call__</code> L735
7	Alphabetize keys and return	<code>__call__</code> L736-738

3. Step 1 — Entry from `generate_report.py`

Inside step **[3/5]** of the orchestrator, a fresh extractor is constructed with the tumor-class label IDs and invoked on the patient-space and MNI-space segmentations. No `map_name` is passed, so the class default `vasari_concise` (L91 of `vasari_auto_v2.py`) is used for the concise translation step.

```
# generate_report.py, step [3/5]
extractor = ExtractVASARI(
    enhancing_label = et_label,
    nonenhancing_label = args.ncr_label,
    oedema_label = args.ed_label,
    verbose = False,
)
vasari_summary = extractor(
    tumorseg_mni = tum_in_mni,
    tumorseg_ss = tumor_path,
    merged = merged_seg,
    metadata = metadata,
)
```

```
metadata.update(vasari_summary) # 'Text Report' lands here
```

4. Step 2 — Raw VASARI features

`extract_vasari_features()` performs all of the voxel/atlas math and assembles a single-row `pandas.DataFrame`. Each column is a VASARI feature; values are mostly small integer codes (e.g. Enhancement Quality $\in \{1,2,3\}$), but a few are raw strings or floats (Tumor Location, Side of Tumor Epicenter, volumes).

```
result = pd.DataFrame(columns=self.COL_NAMES)
result.loc[len(result)] = {
    'Tumor Location': tumor_location,          # raw string, e.g. 'frontal_lobe-cortex region'
    'Side of Tumor Epicenter': side,           # raw string, e.g. 'Midline'
    'Enhancement Quality': enhancement_quality, # int 1/2/3
    'Multifocal or Multicentric': f9_multifocal, # int 1-4
    'Ependymal (ventricular) Invasion': ependymal, # int 1/2
    'Deep WM invasion': deep_wm_f,             # int 1/2
    'CET Crosses midline': CET_cross_midline_f, # int 2/3
    'Asymmetrical Ventricles': asymmetrical_ventricles, # int 0/1
    ...
}
```

5. Step 3 — Concise translation

The raw row is translated by `translate_vasari_report()` using the `vasari_concise` dictionary in `vasari_translation_maps.py`. For each column, the integer code is looked up in the per-feature mapping; if a value isn't in the map (e.g. a raw string), it falls through unchanged via the `try/except` in `translate_row`.

```
def translate_row(row, map_name='vasari'):
    mapping_dict = MAP_STR_TO_MAP[map_name]
    out = row.to_dict()
    for col, mapping in mapping_dict.items():
        if col in out and pd.notna(out[col]):
            val = out[col]
            try:
                out[col] = mapping.get(int(val), val)
            except Exception:
                out[col] = val # raw strings pass through
    return out
```

6. Step 4 — Whitelist filter

Only eight features end up in the Text Report. The list is hard-coded inside `__call__` — some entries (Enlarged Ventricles, Number of lesions, Multiple satellites present, Thickness of enhancing margin) are commented out by the authors.

```
keys_in_text_report = [
    'Tumor Location',
    'Side of Tumor Epicenter',
    'Enhancement Quality',
    'Multifocal or Multicentric',
    'Ependymal (ventricular) Invasion',
    'Deep WM invasion',
    'CET Crosses midline',
    'Asymmetrical Ventricles',
    # 'Enlarged Ventricles',
    # 'Number of lesions',
    # 'Multiple satellites present',
    # 'Thickness of enhancing margin',
```

```
]
result = {k: v for k, v in result.items() if k in keys_in_text_report}
```

Important: the iteration order of `result` is the *original insertion order* from the DataFrame row, not the order of `keys_in_text_report`. That list is used only for filtering.

7. Step 5 — Re-translate with the canonical VASARI map

After step 4, the variable `result` holds the concise-translated subset used to build the sentence. The original `vasari_auto_report` DataFrame is now translated a second time, this time with `map_name='vasari'`, so that every non-Text-Report field in the output dict carries its standard VASARI label (e.g. Enhancement Quality → 'Mild', Asymmetrical Ventricles → 'Absent').

```
vasari_auto_report = self.translate_vasari_report(
    vasari_auto_report, map_name='vasari'
)
```

8. Step 6 — Build the sentence

This is the actual one-liner that produces the Text Report string:

```
vasari_auto_report['Text Report'] = (
    ', '.join(result.values()).capitalize() + '.'
)
```

Three operations happen here:

1. `', '.join(result.values())` concatenates the eight phrases in insertion order, separated by “,”.
2. `.capitalize()` uppercases the first character of the whole string and *lowercases everything else*. This is the reason “Midline” appears as “midline” in the output.
3. A literal “.” is appended as the terminating period.

9. Step 7 — Sort and return

```
vasari_auto_report = dict(sorted(vasari_auto_report.items()))
return vasari_auto_report
```

Alphabetic sorting is purely cosmetic: it controls the position of “Text Report” in the final JSON. The returned dict is merged back into the subject metadata in `generate_report.py`.

10. Worked Example — BraTS2021_01505

The dummy-dataset subject BraTS2021_01505 ships with a populated `*_metadata_final.json`. The table below traces each of the eight whitelisted features from its raw value through the concise translation to its position in the final sentence.

#	Feature	Raw value	After vasari_concise lookup
1	Tumor Location	'frontal_lobe-cortex region' (string)	'frontal_lobe-cortex region' (unchanged – not int)
2	Side of Tumor Epicenter	'Midline' (string)	'Midline' (unchanged – not int)
3	Enhancement Quality	2 (int)	'mildly enhancing'
4	Multifocal or Multicentric	1 (int)	'solitary'
5	Ependymal (ventricular) Invasion	2 (int)	'ependymal invasion present'
6	Deep WM invasion	2 (int)	'deep white matter invasion present'
7	CET Crosses midline	2 (int)	'enhancing region does not cross midline'
8	Asymmetrical Ventricles	0 (int)	'no tumor-associated ventricular asymmetry'

10a. Concatenation

Applying `', '.join(result.values())` over the rows above (in insertion order) yields the intermediate string:

```
frontal_lobe-cortex region, Midline, mildly enhancing, solitary, ependymal invasion present, deep white matter invasion present,
```

10b. Capitalize

`str.capitalize()` upper-cases the first character and lower-cases everything else. Note how the “M” in “Midline” becomes lower-case:

```
Frontal_lobe-cortex region, midline, mildly enhancing, solitary, ependymal invasion present, deep white matter invasion present,
```

10c. Append period

A literal “.” is concatenated to produce the final Text Report value, which matches the one stored in `BraTS2021_01505_metadata_final.json`:

```
Frontal_lobe-cortex region, midline, mildly enhancing, solitary, ependymal invasion present, deep white matter invasion present.
```

11. Caveats and Customization

Subtle behavior of `str.capitalize()`: it lower-cases all characters after the first. Acronyms, proper nouns, or anatomical names embedded in the raw `tumor_location` or `side` strings will be flattened. If preserving case is important, replacing the line with `s[:1].upper() + s[1:] + '.'` keeps the rest of the string untouched.

Adding or removing features: edit `keys_in_text_report` in `__call__`. Re-ordering that list has no effect on output order — the sentence follows DataFrame insertion order, controlled in `extract_vasari_features()`.

Switching translation style: pass a different `map_name` to the constructor (`vasari`, `vasari_sentence`, `vasari_reasoning`, or `vasari_concise` — defined in `vasari_translation_maps.py`). Only the first

translation (line 718) uses this setting; the second translation on line 734 is hard-coded to the canonical `vasari` map.

Where the value lands: the returned dict is alphabetized, then `generate_report.py` calls `metadata.update(vasari_summary)` (line 147). The Text Report is persisted by the final `json.dump(metadata, f, indent=2)` at line 216 of the same file.

12. Source-line Reference Map

File	Lines	Purpose
<code>btreport/generate_report.py</code>	145-147	Instantiate <code>ExtractVASARI</code> and merge result into metadata
<code>btreport/generate_report.py</code>	215-217	Persist final metadata JSON containing Text Report
<code>btreport/vasari_features/vasari_auto_v2.py</code>	91	Default <code>map_name = 'vasari_concise'</code>
<code>btreport/vasari_features/vasari_auto_v2.py</code>	525-541	<code>translate_vasari_report()</code> utility
<code>btreport/vasari_features/vasari_auto_v2.py</code>	543-714	<code>extract_vasari_features()</code> raw computation
<code>btreport/vasari_features/vasari_auto_v2.py</code>	716-738	<code>__call__</code> : translate, filter, build sentence
<code>btreport/vasari_features/vasari_auto_v2.py</code>	735	Line that constructs 'Text Report'
<code>btreport/vasari_features/vasari_translation_maps.py</code>	133-183	<code>VASARI_CONCISE_MAPS</code> (phrases used in Text Report)
<code>btreport/vasari_features/vasari_translation_maps.py</code>	28-57	<code>VASARI_MAPS</code> (canonical labels for other fields)